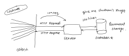
★ Caching and distributed System

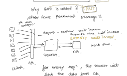
Agenda : Caching
- working
- write policies
- Replacement policies : LRU, LFU and Segmented LFU.

★ CAP Theorem
★ PACELC Theorem.



Types of Storage.

Why RAM is added ? FAST
After have permanent storage ?

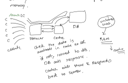


for every req^n, the server will fetch the data from DB.

Basically, we are fetching the data from DB (Permanent Storage).

1st Attempt LC → LC Server → DB
2nd Attempt LC → LC Server → cache.

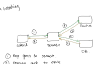
Let try to add a small amount of fast memory.



Cache : - temporary Data store.

Advantages : - improve performance.
- faster Response time.
- Reduce DB loads.

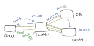
★ Working



① Req goes to server.
② Server ask to cache
③ If data is available in cache,
If data is not available.
④ fetch the data from DB.
⑤ DB respond back to server.
⑥ server responds to clients.

Types of caches :

① Memory cache - Stored in RAM (Redis, Memcached...)
② Disk cache - Stored on disk
eg. Browser cache or OS cache.
③ Web cache - Stores copies of webpages.
eg: (CDN) (akamai, cloudflare)
content delivery network.

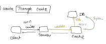


When the requested data is present in cache,
we call it cache hit

when the requested Data is not available on cache
it is cache miss

★ Write policies in caching.

① write Through cache



Whenever write operation is happening,
here, it always happens first on cache
and then it Synced to DB

Advantage : ① Data consistency — cache — DB. will always have the updated data.

② Reads will faster.

Disadvantage :

① Write is slow :
Double work → update cache
→ update DB.

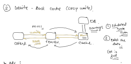
Example : User → updates profile info,
the data will be written to both cache & DB AT THE SAME TIME

```
funtion write (key, value)
{
    cache [key] = value;
    database [key] = value;
}
```

write Through cache.

```
funtion read (key)
{
    if key in cache          check the cache.
    {
        return cache [key];
    }
    else
    {
        value = database [key]     check in DB.
        cache [key] = value;       update the cache.
        return value;
    }
```

② Write - Back cache (Lazy write).



★ Adv :

① write operation are faster (no double work).

② Reduces write req to DB.

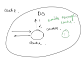
★ Disadv :

① Data Inconsistency [might be]
② Eviction should be configured.

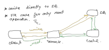
```
fun write (key, value)
{
    cache[key] = value;
    dirty.add (key)          # This is a dirty key
}                            # The key was modified in cache but not in DB.

fun writeback
{
    for key in dirty :
        database [key] = cache [key].
    dirty.clear ;
}
```

③ Write Around cache.



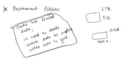
★ write directly to DB.
★ Use cache for only read operation.



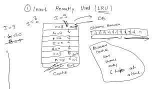
Adv : ① Reduces unnecessary cache writes.
② useful in scenarios where data consistency is not that much important.
→ updated the DP.
→ refresh, it fetching from cache, the old dp.

Disadv : ① This might contain stale (old) data.
② slower Read.

★ Replacement Policies.



① Least Recently Used [LRU]



Adv → Simple to implement.
→ Predictive pattern → (Trending).

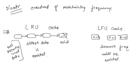
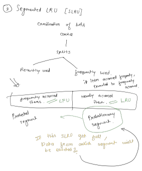
Disadv → maintain a list or Queue.
→ when the Req^n are not Predictive.

② Least frequently used (LFU).
- Replaces the item that is used least frequently.

This cache maintains a frequency counter, for each item, for tracking how many times it is fetched.

Adv will not evict frequently accessed item.

Disadv overload of maintaining frequency.



③ Segmented LRU [SLRU]